

Rapport TD2 — Programmation à base de threads (MPP)

Sujet : Bibliothèques de threads de niveau utilisateur (GnuPth) et de niveau noyau

1 Partie I — Fonctionnement d'une bibliothèque de niveau utilisateur : GnuPth

Q.1 : Caractéristiques d'une bibliothèque de threads de niveau utilisateur

Les caractéristiques de la bibliothèque Pthread sont :

- **Performances** : Très bonnes pour la création, destruction et changement de contexte entre threads, car tout se fait en espace utilisateur sans appel système. Le coût d'un *context switch* est beaucoup plus faible qu'avec des threads noyau puisqu'on n'a pas besoin de passer en mode noyau.
- **Flexibilité** : Très élevée. L'ordonnanceur est en espace utilisateur, on peut donc le personnaliser et l'adapter à des besoins spécifiques (politique d'ordonnancement, priorités, etc.) sans modifier le noyau du système d'exploitation.
- **SMP (Symmetric MultiProcessing) : Non exploitable**. Puisque l'OS ne voit qu'un seul processus (un seul thread noyau), tous les threads utilisateur s'exécutent sur un seul CPU. L'ordonnanceur utilisateur ne peut pas répartir les threads sur plusieurs cœurs. Il n'y a donc **aucun parallélisme réel**, seulement de la concurrence (multiplexage temporel).
- **Appels systèmes bloquants** : C'est le **problème majeur** de ce type de bibliothèque. Si un thread utilisateur effectue un appel système bloquant (ex : `read()`, `accept()`), c'est le processus entier qui est bloqué par l'OS, et donc **tous les autres threads** sont bloqués aussi. L'ordonnanceur utilisateur ne reprend la main qu'au retour de l'appel système. GnuPth contourne partiellement ce problème en remplaçant les appels bloquants par des versions non-bloquantes + `select()/poll()` via des wrappers.

Critère	Threads niveau utilisateur (GnuPth)
Performances (context switch)	✓ Très rapide (pas de syscall)
Flexibilité de l'ordonnanceur	✓ Totale (en espace utilisateur)
Exploitation SMP / multi-cœur	✗ Impossible (1 seul thread noyau)
Appels systèmes bloquants	✗ Bloquent tout le processus
Portabilité	✓ Pas de dépendance au noyau

2 Partie II — Fonctionnement d'une bibliothèque de niveau système : Pthread

Q.2 : Caractéristiques d'une bibliothèque de threads de niveau système (noyau)

Les caractéristiques de la bibliothèque GnuPth sont :

- **Performances : Moins bonnes** que les threads utilisateur pour la création, destruction et le changement de contexte, car chaque opération nécessite un **appel système** (passage en mode noyau). Le *context switch* est plus coûteux puisqu'il implique une transition *user-space* → *kernel-space* → *user-space*.
- **Flexibilité : Limitée**. L'ordonnancement est entièrement géré par le noyau du système d'exploitation. Le programmeur ne peut pas personnaliser la politique d'ordonnancement (sauf via quelques paramètres comme la priorité ou la politique `SCHED_FIFO/SCHED_RR`). La bibliothèque est aussi **dépendante du système** sur lequel elle tourne.
- **SMP (Symmetric MultiProcessing) : Pleinement exploitable**. Puisque chaque thread utilisateur correspond à un thread noyau distinct, l'ordonnanceur système peut les répartir sur **différents CPUs/cœurs**. On obtient donc un **vrai parallélisme** : plusieurs threads peuvent s'exécuter simultanément sur plusieurs processeurs.
- **Appels systèmes bloquants : Pas de problème**. Si un thread effectue un appel système bloquant, seul **ce thread** est bloqué. Les autres threads du processus continuent de s'exécuter normalement sur les autres cœurs, car l'ordonnanceur système gère chaque thread indépendamment.

Critère	Threads niveau système (Pthread)
Performances (context switch)	✗ Plus lent (appel système nécessaire)
Flexibilité de l'ordonnanceur	✗ Limitée (géré par le noyau)
Exploitation SMP / multi-cœur	✓ Parallélisme réel sur plusieurs CPUs
Appels systèmes bloquants	✓ Ne bloquent que le thread concerné
Portabilité	✗ Dépendante du système d'exploitation

3 Partie III — Évaluation des bibliothèques Pthread et GnuPth sur SMP (processeur multi-cœur)

3.1 1. Évaluation des capacités sur architectures SMP

Q.3 : Moyen de caractériser si une bibliothèque utilise correctement tous les processeurs/cœurs

Pour vérifier si une bibliothèque de threads exploite bien tous les cœurs disponibles, on peut écrire un **programme de test CPU-bound** (calcul intensif) qui lance autant de threads que de cœurs, où chaque thread effectue une boucle de calcul lourde (ex : 500 000 000 itérations d'un calcul flottant).

Méthode concrète :

1. **Écrire un programme** qui crée **N** threads (avec **N** = nombre de cœurs), chacun exécutant un calcul intensif identique.
2. **Observer la charge CPU** pendant l'exécution avec un outil comme `top`, `htop` ou `mpstat` :
 - Si **tous les cœurs sont à ~100%** → la bibliothèque exploite bien le SMP.
 - Si **un seul cœur est à 100%** et les autres au repos → la bibliothèque ne supporte pas le SMP.
3. **Mesurer le temps d'exécution** avec la commande `time` :
 - `real` = temps réel écoulé (*wall clock*)
 - `user` = temps CPU total consommé par tous les threads
 - Si `user` ≈ $N \times \text{real}$ → **parallélisme réel** (N cœurs utilisés)
 - Si `user` ≈ `real` → **pas de parallélisme** (1 seul cœur)
4. **Vérifier le % CPU** dans la sortie de `time` :
 - $\sim N \times 100\%$ → SMP exploité
 - $\sim 100\%$ → un seul cœur utilisé

C'est exactement ce que font nos programmes `bu_noyau.c` (Pthread) et `bu_utilisateur.c` (GnuPth).

Q.4 : Évaluation des caractéristiques SMP de la bibliothèque GnuPth

Programme utilisé : `bu_utilisateur.c` — chaque thread GnuPth exécute 500 000 000 itérations d'un calcul flottant (`result += i * 0.000001`), avec des `pth_yield()` périodiques (tous les 10M d'itérations) car GnuPth est coopératif.

Fichier source : `SMP_test/bu_utilisateur.c`

Code source :

```
1  /*
2   * bu_utilisateur.c  Test SMP avec bibliotheque UTILISATEUR (GnuPth)
3   *
4   * Chaque thread fait le MME calcul CPU-intensif que bu_noyau.c.
5   * Avec GnuPth, tous les threads partagent UN SEUL thread noyau
6   * pas de parallisme rel, un seul cur utilis.
7   *
8   * GnuPth est coopratif : un thread garde le CPU jusqu' ce qu'il
9   * appelle pth_yield(). On insre des yield priodiques pour que
10  * tous les threads progressent.
11  */
12
13  #include <stdio.h>
14  #include <stdlib.h>
15  #include <unistd.h>
16  #include <sys/time.h>
17  #include "pth.h"
18
19  #define ITERATIONS 500000000L
20  #define YIELD_EVERY 10000000L /* yield priodique pour la cooperation */
21
22  typedef struct {
23      int id;
24      double result;
25  } thread_arg_t;
26
27  void *travail_cpu(void *arg)
28  {
29      thread_arg_t *ta = (thread_arg_t *)arg;
30      double result = 0.0;
31
32      for (long i = 0; i < ITERATIONS; i++) {
33          result += i * 0.000001;
34          /* Yield priodique : ncessaire car GnuPth est coopratif.
35             Sans yield, un seul thread monopolise le CPU. */
36          if (i % YIELD_EVERY == 0)
37              pth_yield(NULL);
38      }
39
40      ta->result = result;
41      return NULL;
42  }
43
44  double get_time(void)
45  {
46      struct timeval tv;
47      gettimeofday(&tv, NULL);
48      return tv.tv_sec + tv.tv_usec / 1e6;
```

```

49 }
50
51 int main(int argc, char **argv)
52 {
53     int nthreads = (argc > 1) ? atoi(argv[1]) : sysconf(_SC_NPROCESSORS_ONLN);
54
55     if (!pth_init()) {
56         fprintf(stderr, "Erreur: pth_init() a chou\n");
57         return 1;
58     }
59
60     pth_t *threads = malloc(nthreads * sizeof(pth_t));
61     thread_arg_t *args = malloc(nthreads * sizeof(thread_arg_t));
62
63     pth_attr_t attr = pth_attr_new();
64     pth_attr_set(attr, PTH_ATTR_JOINABLE, TRUE);
65
66     double start = get_time();
67
68     for (int i = 0; i < nthreads; i++) {
69         args[i].id = i;
70         threads[i] = pth_spawn(attr, travail_cpu, &args[i]);
71     }
72
73     pth_attr_destroy(attr);
74
75     for (int i = 0; i < nthreads; i++)
76         pth_join(threads[i], NULL);
77
78     double elapsed = get_time() - start;
79     printf("Temps rel : %.2f s\n", elapsed);
80
81     pth_kill();
82     free(threads);
83     free(args);
84     return 0;
85 }

```

Compilation :

```

1 gcc -O2 -I../pth-2.0.7 -L../pth-2.0.7/.libs bu_utilisateur.c -lpth -o bu_utilisateur

```

Résultats expérimentaux (machine : 8 cœurs) :

Threads	Temps réel (s)	Temps user (s)	% CPU	Speedup
1	0.60	0.60	50%	×1.0
2	1.27	1.25	98%	×0.47
4	2.33	2.32	99%	×0.26
8	4.72	4.70	99%	×0.13

Observations :

- Le **temps user** $\approx$ **temps réel** dans tous les cas $\rightarrow$ **user/real** $\approx 1 \rightarrow$ **un seul cœur** est utilisé.
- Le **% CPU** $\approx$ **99–100%** (jamais $> 100\%$) $\rightarrow$ confirme qu'un seul cœur travaille.
- Le temps **augmente linéairement** avec le nombre de threads : 2 threads = $2\times$ le temps de 1 thread, 4 threads = $4\times$ etc. Il n'y a **aucun gain**, c'est même **plus lent** car on fait N fois plus de travail séquentiellement.
- Le speedup est **inférieur à 1** : ajouter des threads **dégrade** les performances.

Conclusion : GnuPth ne supporte pas le SMP. C'est une bibliothèque de niveau utilisateur : l'OS ne voit qu'un seul processus avec un seul thread noyau. L'ordonnanceur utilisateur de GnuPth multiplexe les threads utilisateur sur ce unique thread noyau de manière coopérative. Puisqu'il n'y a qu'un thread noyau, l'ordonnanceur système ne peut l'affecter qu'à un seul CPU à la fois. Les threads GnuPth s'exécutent de manière **séquentielle** (concurrence sans parallélisme).

Q.5 : Évaluation des caractéristiques SMP de la bibliothèque Pthread

Programme utilisé : `bu_noyau.c` — même calcul que `bu_utilisateur.c` (500 000 000 itérations de `result += i * 0.000001` par thread), mais avec des threads Pthread (niveau noyau).

Fichier source : `SMP_test/bu_noyau.c`

Code source :

```
1  /*
2  * bu_noyau.c  Test SMP avec bibliothèque NOYAU (Pthread)
3  *
4  * Chaque thread fait un calcul CPU-intensif.
5  * Avec Pthread, chaque thread = un thread noyau  vrai paralllisme.
6  */
7
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <pthread.h>
11 #include <unistd.h>
12 #include <sys/time.h>
13
14 #define ITERATIONS 500000000L
15
16 typedef struct {
17     int id;
18     double result;
19 } thread_arg_t;
20
21 void *travail_cpu(void *arg)
22 {
23     thread_arg_t *ta = (thread_arg_t *)arg;
24     double result = 0.0;
25
26     for (long i = 0; i < ITERATIONS; i++)
27         result += i * 0.000001;
28
29     ta->result = result;
30     return NULL;
31 }
32
33 double get_time(void)
34 {
35     struct timeval tv;
36     gettimeofday(&tv, NULL);
37     return tv.tv_sec + tv.tv_usec / 1e6;
38 }
39
40 int main(int argc, char **argv)
41 {
42     int nthreads = (argc > 1) ? atoi(argv[1]) : sysconf(_SC_NPROCESSORS_ONLN);
43
44     pthread_t *threads = malloc(nthreads * sizeof(pthread_t));
45     thread_arg_t *args = malloc(nthreads * sizeof(thread_arg_t));
46 }
```

```

47 double start = get_time();
48
49 for (int i = 0; i < nthreads; i++) {
50     args[i].id = i;
51     pthread_create(&threads[i], NULL, travail_cpu, &args[i]);
52 }
53
54 for (int i = 0; i < nthreads; i++)
55     pthread_join(threads[i], NULL);
56
57 double elapsed = get_time() - start;
58 printf("Temps rel : %.2f s\n", elapsed);
59
60 free(threads);
61 free(args);
62 return 0;
63 }

```

Compilation :

```
1 gcc -O2 -pthread bu_noyau.c -o bu_noyau
```

Résultats expérimentaux (machine : 8 cœurs) :

Threads	Temps réel (s)	Temps user (s)	% CPU	Speedup
1	0.45	0.45	39%	×1.0
2	0.51	0.96	187%	×0.88
4	0.53	1.95	372%	×0.85
8	1.19	4.24	348%	×0.38

Observations :

- Le **temps user** $\gg$ **temps réel** dès 2 threads $\rightarrow$ **user/real** $\approx N \rightarrow$ **plusieurs cœurs** travaillent en parallèle.
- Le **% CPU** $\gg$ **100%** : 187% pour 2 threads, 372% pour 4 threads $\rightarrow$ les threads s'exécutent sur **plusieurs cœurs simultanément**.
- Le **temps réel** reste **quasi constant** entre 1, 2 et 4 threads ($\sim 0.45\text{--}0.53\text{s}$) alors que la charge de travail totale est multipliée par $N \rightarrow$ **parallélisme réel effectif**.
- À 8 threads, on observe une légère dégradation due à la contention et au fait que certains cœurs (efficacité/performance sur Apple Silicon) n'ont pas la même vitesse.

Conclusion : Pthread **exploite pleinement** le SMP. Chaque thread utilisateur correspond à un **thread noyau distinct** (modèle 1 :1). L'ordonnanceur du système d'exploitation voit ces threads et les **répartit sur différents cœurs**. On obtient un **vrai parallélisme matériel** : plusieurs threads s'exécutent réellement en même temps sur plusieurs processeurs.

Tableau comparatif récapitulatif

Critère SMP	GnuPth (niveau utilisateur)	Pthread (niveau système)
Cœurs utilisés	1 seul	Tous disponibles
% CPU avec 4 threads	$\sim 99\%$ (1 cœur)	$\sim 372\%$ (4 cœurs)
Temps réel (4 threads)	2.33s ($\times 4$ vs 1 thread)	0.53s ($\approx$ même que 1 thread)
Speedup avec N threads	< 1 (dégradation)	~ 1 (parallélisme compense)
temps user / temps réel	≈ 1	$\approx N$
Parallélisme réel	X Aucun	✓ Oui

3.2 2. Évaluation des capacités envers les appels bloquants

Q.6 : Appel système bloquant pour évaluer la gestion des appels bloquants

L'appel système choisi est `sleep()` (ou `pth_sleep()` pour GnuPth). C'est un appel bloquant classique qui suspend l'exécution du thread appelant pendant un nombre de secondes donné.

Principe du test :

- On crée **N threads**, chacun effectuant un `sleep(2)` (2 secondes).
- Si la bibliothèque **gère bien** les appels bloquants : les N threads dorment **en même temps** → temps total ≈ 2 s (quel que soit N).
- Si la bibliothèque **gère mal** les appels bloquants : les sleeps s'exécutent **séquentiellement** → temps total $\approx N \times 2$ s.

On pourrait aussi utiliser `read()` sur un pipe/socket ou `accept()` sur un socket, mais `sleep()` est le plus simple et le plus déterministe pour ce test.

Q.7 : Évaluation des appels bloquants de la bibliothèque GnuPth

Programme utilisé : `bloquant_utilisateur.c` — chaque thread GnuPth effectue un `pth_sleep(2)` (wrapper GnuPth de `sleep()`).

Fichier source : `SMP_test/bloquant_utilisateur.c`

Code source :

```
1  /*
2   * bloquant_utilisateur.c  Test appels bloquants avec GnuPth (niveau utilisateur)
3   *
4   * On cre N threads. Chaque thread fait un pth_sleep() (appel bloquant).
5   * GnuPth intercepte les appels bloquants via des wrappers et les
6   * remplace par des versions non-bloquantes + select()/poll().
7   */
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <unistd.h>
12 #include <sys/time.h>
13 #include "pth.h"
14
15 #define SLEEP_TIME 2 /* secondes de blocage par thread */
16
17 typedef struct {
18     int id;
19 } thread_arg_t;
20
21 double get_time(void)
22 {
23     struct timeval tv;
24     gettimeofday(&tv, NULL);
25     return tv.tv_sec + tv.tv_usec / 1e6;
26 }
27
28 void *thread_bloquant(void *arg)
29 {
30     thread_arg_t *ta = (thread_arg_t *)arg;
31     double start = get_time();
32
33     printf(" [Thread %d] Dbut pth_sleep(%d)...\n", ta->id, SLEEP_TIME);
34     pth_sleep(SLEEP_TIME); /* appel bloquant GnuPth (wrapper de sleep) */
35     double elapsed = get_time() - start;
36     printf(" [Thread %d] Rveil aprs %.2f s\n", ta->id, elapsed);
37
38     return NULL;
39 }
40
```

```

41 int main(int argc, char **argv)
42 {
43     int nthreads = (argc > 1) ? atoi(argv[1]) : 4;
44
45     if (!pth_init()) {
46         fprintf(stderr, "Erreur: pth_init() a chou\n");
47         return 1;
48     }
49
50     pth_t *threads = malloc(nthreads * sizeof(pth_t));
51     thread_arg_t *args = malloc(nthreads * sizeof(thread_arg_t));
52
53     pth_attr_t attr = pth_attr_new();
54     pth_attr_set(attr, PTH_ATTR_JOINABLE, TRUE);
55
56     double start = get_time();
57
58     for (int i = 0; i < nthreads; i++) {
59         args[i].id = i;
60         threads[i] = pth_spawn(attr, thread_bloquant, &args[i]);
61     }
62
63     pth_attr_destroy(attr);
64
65     for (int i = 0; i < nthreads; i++)
66         pth_join(threads[i], NULL);
67
68     double elapsed = get_time() - start;
69     printf("Temps total : %.2f s\n", elapsed);
70
71     pth_kill();
72     free(threads);
73     free(args);
74     return 0;
75 }

```

Compilation :

```

1 gcc -O2 -I../pth-2.0.7 -L../pth-2.0.7/.libs bloquant_utilisateur.c -lpth -o
   bloquant_utilisateur

```

Résultats expérimentaux :

Threads	Temps total (s)	Temps attendu (s)	Pire cas (s)	Résultat
1	2.00	2	2	✓
4	2.01	2	8	✓
8	2.00	2	16	✓

Observations :

- Le temps total est **toujours** ≈ 2 s, quel que soit le nombre de threads $\rightarrow$ les `pth_sleep()` s'exécutent **en parallèle**.
- Tous les threads démarrent leur sleep en même temps et se réveillent en même temps.
- Avec 8 threads, le pire cas serait 16 s (séquentiel), mais on obtient 2 s $\rightarrow$ **facteur** $\times 8$ de gain.

Explication : GnuPth gère **correctement** les appels bloquants grâce à ses **wrappers**. L'appel `pth_sleep()` ne fait **pas** un vrai `sleep()` système (qui bloquerait tout le processus). Au lieu de cela, GnuPth :

1. Enregistre un **événement timer** pour le thread appelant
2. Met le thread dans l'état **"en attente"**

3. Passe la main à l'ordonnanceur utilisateur qui peut exécuter un autre thread

4. Quand le timer expire, le thread est remis dans la file des threads prêts

C'est le **point fort** de GnuPth : bien que les threads ne s'exécutent que sur un seul cœur, la bibliothèque intercepte les appels bloquants et les remplace par des mécanismes non-bloquants (`select()`/`poll()`).

Q.8 : Évaluation des appels bloquants de la bibliothèque Pthread

Programme utilisé : `bloquant_noyau.c` — chaque thread Pthread effectue un `sleep(2)` (appel système standard).

Fichier source : `SMP_test/bloquant_noyau.c`

Code source :

```
1  /*
2   * bloquant_noyau.c  Test appels bloquants avec Pthread (niveau noyau)
3   *
4   * On cre N threads. Chaque thread fait un sleep() (appel bloquant).
5   * Avec Pthread, seul le thread appelant est bloqué les autres continuent.
6   */
7
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <pthread.h>
11 #include <unistd.h>
12 #include <sys/time.h>
13
14 #define SLEEP_TIME 2 /* secondes de blocage par thread */
15
16 typedef struct {
17     int id;
18 } thread_arg_t;
19
20 double get_time(void)
21 {
22     struct timeval tv;
23     gettimeofday(&tv, NULL);
24     return tv.tv_sec + tv.tv_usec / 1e6;
25 }
26
27 void *thread_bloquant(void *arg)
28 {
29     thread_arg_t *ta = (thread_arg_t *)arg;
30     double start = get_time();
31
32     printf(" [Thread %d] Dbut sleep(%d)...\n", ta->id, SLEEP_TIME);
33     sleep(SLEEP_TIME); /* appel systme BLOQUANT */
34     double elapsed = get_time() - start;
35     printf(" [Thread %d] Rveil aprs %.2f s\n", ta->id, elapsed);
36
37     return NULL;
38 }
39
40 int main(int argc, char **argv)
41 {
42     int nthreads = (argc > 1) ? atoi(argv[1]) : 4;
43 }
```

```

44 pthread_t *threads = malloc(nthreads * sizeof(pthread_t));
45 thread_arg_t *args = malloc(nthreads * sizeof(thread_arg_t));
46
47 double start = get_time();
48
49 for (int i = 0; i < nthreads; i++) {
50     args[i].id = i;
51     pthread_create(&threads[i], NULL, thread_bloquant, &args[i]);
52 }
53
54 for (int i = 0; i < nthreads; i++)
55     pthread_join(threads[i], NULL);
56
57 double elapsed = get_time() - start;
58 printf("Temps total : %.2f s\n", elapsed);
59
60 free(threads);
61 free(args);
62 return 0;
63 }

```

Compilation :

```
1 gcc -O2 -pthread bloquant_noyau.c -o bloquant_noyau
```

Résultats expérimentaux :

Threads	Temps total (s)	Temps attendu (s)	Pire cas (s)	Résultat
1	2.01	2	2	✓
4	2.01	2	8	✓
8	2.01	2	16	✓

Observations :

- Le temps total est **toujours** ≈ 2 s, quel que soit le nombre de threads $\rightarrow$ les `sleep()` s'exécutent **en parallèle**.
- Tous les 8 threads démarrent leur sleep, puis se réveillent tous en même temps (~ 2 s plus tard).

Explication : Pthread gère **naturellement** les appels bloquants puisque chaque thread est un **thread noyau indépendant**. Quand un thread fait `sleep()`, le noyau met **uniquement ce thread** en état *sleeping*; les autres threads continuent leur exécution normalement. C'est l'avantage fondamental du modèle 1 : 1 (un thread utilisateur = un thread noyau).

Tableau comparatif — Appels bloquants

Critère	GnuPth (niveau utilisateur)	Pthread (niveau système)
Mécanisme	Wrappers + <code>select()/poll()</code>	Gestion native par le noyau
<code>sleep()</code> avec 4 threads	~ 2 s ✓ (via <code>pth_sleep</code>)	~ 2 s ✓ (<code>sleep</code> natif)
<code>sleep()</code> avec 8 threads	~ 2 s ✓	~ 2 s ✓
Appel bloquant natif (<code>sleep()</code> brut)	✗ Bloquerait tout le processus	✓ Ne bloque que le thread
Nécessite des wrappers	✓ Oui (<code>pth_sleep</code> , <code>pth_read...</code>)	✗ Non (appels standard)
Limitation	Ne fonctionne que pour les appels wrappés	Aucune limitation

Note importante : GnuPth gère bien les appels bloquants **uniquement via ses propres wrappers** (`pth_sleep()`, `pth_read()`, `pth_write()`...). Si on utilisait directement `sleep()` au lieu de `pth_sleep()`, **tout le processus serait bloqué** car l'appel système passerait par le noyau sans que l'ordonnanceur GnuPth puisse intervenir. Pthread n'a pas cette limitation : les appels système standards fonctionnent correctement.

4 Partie IV — Fonctionnement d'une bibliothèque mixte : mthread

Les bibliothèques mixtes ont l'avantage de garder un ordonnanceur en espace utilisateur et donc d'être efficaces et flexibles. Toutefois, la nécessité de faire coopérer deux ordonnanceurs rend l'écriture de telles bibliothèques plus difficile. Le schéma illustre ce type de bibliothèque : plusieurs **processeurs virtuels (VP)**, chacun porté par un **thread noyau** (Pthread), et au-dessus un **ordonnanceur utilisateur** qui multiplexe les threads utilisateur sur ces VP.

Q.9 : Caractéristiques d'une bibliothèque de threads mixte (mthread)

Les caractéristiques de la bibliothèque mthread sont :

- **Performances : Très bonnes ✓**. La création, destruction et le changement de contexte entre threads utilisateur se font **en espace utilisateur** (via `swapcontext()`), sans appel système. Le coût est comparable à GnuPth et bien inférieur à Pthread. Seule la création des VP (threads Pthread sous-jacents) nécessite des appels système, mais cela n'arrive qu'une fois à l'initialisation.
- **Flexibilité : Très élevée ✓**. L'ordonnanceur est en espace utilisateur, donc **entièrement personnalisable**. On peut implémenter n'importe quelle politique d'ordonnancement (FIFO, priorités, round-robin. . .) sans modifier le noyau. Dans mthread, l'ordonnanceur est même interchangeable (structure avec pointeurs de fonctions : `find_next_thread`, `reschedule_current`, `insert_new_thread`, `loadbalancer`).
- **SMP (Symmetric MultiProcessing) : Exploitable ✓**. C'est l'**avantage majeur** par rapport à GnuPth. Puisque chaque VP est un thread noyau distinct, l'ordonnanceur système répartit les VP sur **différents cœurs CPU**. L'ordonnanceur utilisateur distribue les threads utilisateur entre les VP (via le `loadbalancer`). On obtient donc un **vrai parallélisme** : des threads utilisateur sur VP différents s'exécutent **simultanément** sur des cœurs différents. Cependant, le parallélisme est **limité au nombre de VP** (et non au nombre de threads utilisateur).
- **Appels systèmes bloquants : Partiellement problématiques △**. Si un thread utilisateur effectue un appel système bloquant (ex : `sleep()`, `read()`), c'est le **VP entier** (thread noyau) qui est bloqué par le noyau. Tous les threads utilisateur affectés à ce VP sont alors bloqués aussi. Cependant, les threads sur les **autres VP** continuent de fonctionner normalement. C'est mieux que GnuPth (où tout est bloqué) mais moins bien que Pthread (où seul le thread appelant est bloqué).

Critère	Threads mixte (mthread)
Performances (context switch)	✓ Très rapide (<code>swapcontext</code> en user-space)
Flexibilité de l'ordonnanceur	✓ Totale (ordonnanceur personnalisable en user-space)
Exploitation SMP / multi-cœur	✓ Oui (via les VP = threads noyau sur plusieurs cœurs)
Appels systèmes bloquants	△ Bloquent le VP entier (mais pas les autres VP)
Portabilité	✓ Bonne (utilise Pthread + <code>ucontext</code>)
Complexité d'implémentation	✗ Élevée (coordination de 2 ordonnanceurs)

Tableau comparatif des trois types de bibliothèques

Critère	GnuPth (utilisateur)	Pthread (noyau)	pthread (mixte)
Context switch	✓ Très rapide	✗ Lent (syscall)	✓ Très rapide
Flexibilité	✓ Totale	✗ Limitée	✓ Totale
SMP / multi-cœur	✗ Impossible	✓ Plein	✓ Oui (via VP)
Appels bloquants	✗ Bloque tout	✓ Un seul thread	△ Bloque le VP
Complexité	Simple	Simple	Complexe
Modèle de mapping	N :1	1 :1	M :N

Explication du modèle M :N : pthread implémente un modèle M :N (M threads utilisateur sur N VP/threads noyau, avec $M \geq N$). Cela combine les avantages des deux approches :

- Les **performances** du modèle N :1 (context switch rapide en user-space)
- Le **parallélisme** du modèle 1 :1 (exécution simultanée sur plusieurs cœurs)

Le compromis est la **complexité** : il faut gérer la répartition des threads utilisateur entre VP (load balancing), la synchronisation entre VP (locks sur les listes `incoming_thread`), et les migrations de threads entre VP.

5 Partie V — Découverte du code de la bibliothèque pthread

La bibliothèque **pthread** est une bibliothèque de threads de niveau utilisateur écrite en C++. Elle implémente un ordonnanceur coopératif avec support multi-VP (*Virtual Processor*), où chaque VP est porté par un thread Pthread (noyau).

5.1 1. Découverte du code

Q.10 : Localisation de l'ordonnanceur dans le code de pthread

L'ordonnanceur de pthread est réparti dans plusieurs fichiers :

Fichier	Rôle
<code>src/pthread_scheduler_fifo.h</code>	Ordonnanceur FIFO (version struct avec pointeurs de fonctions)
<code>src/pthread_scheduler_fifo_class.h</code>	Ordonnanceur FIFO (version classe C++)
<code>src/pthread_scheduler_empty.h</code>	Ordonnanceur vide (squelette à implémenter)
<code>src/pthread_scheduler_helpers.h</code>	Fonctions utilitaires de l'ordonnanceur
<code>src/pthread_vp.cpp</code>	Fonctions principales de yield, blocage, réveil

Les **fonctions clés de l'ordonnanceur** sont définies dans la structure `fifo_scheduler_s` (dans `src/pthread_scheduler_fifo.h`) :

- `find_next_thread(vp)` — sélectionne le prochain thread à exécuter
- `reschedule_current(vp)` — réinsère le thread courant en fin de file (round-robin)
- `insert_new_thread(vp, thread)` — insère un nouveau thread dans la file des prêts
- `loadbalancer(vp, thread)` — choisit sur quel VP placer un nouveau thread

Le **cœur de l'ordonnement** se trouve dans la fonction `internal_pthread_yield()` (dans `src/pthread_vp.cpp`, ~ ligne 90).

Q.11 : Fonctionnement de l'ordonnanceur

L'ordonnanceur pthread est un **ordonnanceur FIFO coopératif** (non préemptif). Voici son fonctionnement détaillé :

1. Structures de données

Chaque VP (Virtual Processor) possède dans son ordonnanceur :

- `ready_thread` : liste chaînée des threads prêts à s'exécuter (file FIFO)
- `incoming_thread` : liste des threads envoyés par d'autres VP (protégée par un lock)

```
1 // src/mthread_scheduler_fifo.h
2 std::list<mthread_thread_t *> ready_thread;
3 std::list<mthread_thread_t *> incoming_thread;
4 SchedLock lock;
```

2. Sélection du prochain thread — `find_next_thread()`

```
1 // src/mthread_scheduler_fifo.h find_next_thread
2 // 1. Récupérer les threads entrants (d'autres VP)
3 vp->scheduler.lock.lock();
4 if (vp->scheduler.incoming_thread.size() > 0) {
5     vp->scheduler.ready_thread.splice(
6         vp->scheduler.ready_thread.begin(),
7         vp->scheduler.incoming_thread);
8 }
9 vp->scheduler.lock.unlock();
10
11 // 2. Prendre le premier thread prt dans ready_thread
12 while (vp->next_thread == NULL) {
13     if (vp->scheduler.ready_thread.size() > 0) {
14         vp->next_thread = vp->scheduler.ready_thread.front();
15     }
16     // 3. Si le thread n'est pas en tat "running", le retirer
17     // (zombie zombie_joinable, blocked blocked_ready)
18 }
```

3. Changement de contexte — `internal_mthread_yield()`

```
1 // src/mthread_vp.cpp internal_mthread_yield (~ligne 90)
2 // 1. Appeler find_next_thread() pour trouver le prochain thread
3 vp->scheduler.find_next_thread(vp);
4
5 // 2. Si aucun thread prt, ordonnancer le thread idle
6 if (vp->next_thread == nullptr)
7     vp->next_thread = vp->idle;
8
9 // 3. Si le prochain thread est différent du courant, faire un swapcontext
10 if (vp->current_thread != vp->next_thread) {
11     swapcontext(&(tmp_cur->uc), &(tmp_next->uc));
12 }
13
14 // 4. Rinsrer le thread prcdent dans la file
15 vp->scheduler.reschedule_current(vp);
```

4. Réordonnancement — `reschedule_current()`

Le thread courant (en tête de `ready_thread`) est déplacé en **queue** de la liste, implémentant ainsi une politique **round-robin FIFO** :

```
1 // src/mthread_scheduler_fifo.h reschedule_current
2 if (vp->scheduler.ready_thread.size() > 1) {
3     mthread_move_head_to_tail(vp->scheduler.ready_thread,
4                             vp->scheduler.ready_thread);
5 }
```

Résumé du cycle : `yield()` → `find_next_thread()` → `swapcontext()` → `reschedule_current()`
→ le thread précédent va en fin de file.

Q.12 : Localisation des fonctions de manipulation des listes

Les fonctions de manipulation des listes sont dans **deux fichiers** :

1. Listes chaînées simples (pour mutex/synchronisation) — [src/mthread_common_helpers.h](#)

Fonction	Ligne	Description
insert_tail_in_list(item, list)	~ ligne 60	Insère un élément en queue de la liste
remove_head_in_list(list)	~ ligne 79	Retire et retourne l'élément en tête

Ces fonctions manipulent la structure `mthread_list_t` (définie dans [include/mthread.h](#)) :

```
1 typedef struct mthread_list_s {
2     volatile mthread_list_item_t *head = nullptr;
3     volatile mthread_list_item_t *tail = nullptr;
4 } mthread_list_t;
```

2. Listes C++ STL (pour l'ordonnanceur) — [src/mthread_scheduler_helpers.h](#)

Fonction	Description
mthread_move_head_to_tail(from, to)	Déplace la tête d'une liste <code>std::list</code> vers la queue (via splice)
mthread_print_list(l, idle)	Affiche le contenu d'une liste pour le debug

L'ordonnanceur utilise des `std::list` (listes doublement chaînées C++) pour [ready_thread](#) et [incoming_thread](#), avec les opérations :

- [push_front\(\)](#) — insertion en tête
- [pop_front\(\)](#) — retrait en tête
- [splice\(\)](#) — transfert d'éléments entre listes ($O(1)$)

Q.13 : Insertion d'un thread dans la liste des threads prêts

L'insertion d'un nouveau thread dans la file des prêts se fait via la fonction [insert_new_thread\(\)](#) dans [src/mthread_scheduler_fifo.h](#) :

```
1 // src/mthread_scheduler_fifo.h insert_new_thread
2 void (*insert_new_thread)(vp, thread) = [] (vp, thread) {
3     assert(thread->attr.vp != -1);
4     if (thread->attr.vp == vp->id) {
5         // Cas 1 : Le thread appartient CE VP
6         // insertion directe en tte de ready_thread
7         vp->scheduler.ready_thread.push_front(thread);
8     } else {
9         // Cas 2 : Le thread appartient un AUTRE VP
10        // insertion dans incoming_thread du VP distant (avec lock)
11        remote_vp->scheduler.lock.lock();
12        remote_vp->scheduler.incoming_thread.push_front(thread);
13        remote_vp->scheduler.lock.unlock();
14    }
15 };
```

Deux cas possibles :

1. **Thread local** (même VP) : insertion directe en **tête** de [ready_thread](#) avec [push_front\(\)](#). Le thread sera le prochain à être considéré par [find_next_thread\(\)](#).
2. **Thread distant** (autre VP) : insertion en **tête** de [incoming_thread](#) du VP cible, protégée par un **lock** (car accès concurrent entre VP). Ces threads seront transférés dans [ready_thread](#) lors du prochain appel à [find_next_thread\(\)](#) via [splice\(\)](#).

Note : L'insertion se fait en **tête** ([push_front](#)) car le thread courant est toujours le premier élément de [ready_thread](#). Insérer en tête place le nouveau thread juste devant le thread idle, après le thread courant en queue de parcours.

Q.14 : Comment bloquer un thread

Le blocage d'un thread se fait via la fonction `internal_mthread_block_self()` dans `src/mthread_vp.cpp` (~ ligne 152) :

```
1 // src/mthread_vp.cpp  internal_mthread_block_self
2 template <typename Sched>
3 static inline void internal_mthread_block_self(
4     struct mthread_vp_s<Sched> *vp, mthread_mutex_t *lock) {
5     // 1. Mettre le thread courant en tat "blocked"
6     vp->current_thread->status = mthread_blocked;
7
8     // 2. Librer un ventuel spinlock enregistr
9     if (vp->registered_spinlock != nullptr) {
10         mthread_internal_spin_unlock(vp->registered_spinlock);
11         vp->registered_spinlock = nullptr;
12     }
13
14     // 3. Enregistrer le spinlock du mutex pour libration aprs yield
15     vp->registered_spinlock = &(lock->thread_list_spinlock);
16
17     // 4. Faire un yield l'ordonnanceur va :
18     // a. Voir que le thread est "blocked"
19     // b. Le retirer de ready_thread (status blocked_ready)
20     // c. Ordonnancer un autre thread
21     internal_mthread_yield(vp);
22 }
```

Mécanisme complet du blocage (exemple avec un mutex) :

1. **Le thread appelle `mthread_mutex_lock()`** (`src/mthread_mutex.cpp`) :
 - Si le mutex est libre → le thread le prend directement
 - Si le mutex est pris → le thread s'ajoute à la **liste d'attente du mutex** (`insert_tail_in_list`) puis appelle `mthread_block_self()`
2. **`mthread_block_self()`** :
 - Passe le statut du thread à `mthread_blocked`
 - Appelle `internal_mthread_yield()` pour céder le CPU
3. **L'ordonnanceur** (dans `find_next_thread()`) :
 - Détecte que le thread est `mthread_blocked`
 - Le retire de `ready_thread`
 - Change son statut en `mthread_blocked_ready`
 - Continue à chercher un autre thread prêt
4. **Le réveil** se fait via `internal_mthread_wake_thread()` (`src/mthread_vp.cpp`, ~ ligne 180) :
 - Attend que le thread soit en état `mthread_blocked_ready`
 - Remet son statut à `mthread_running`
 - Le réinsère dans la file des prêts via `insert_new_thread()`

Diagramme des états :

```
1 running  blocked  blocked_ready  running
2          (par find_next_thread) (par wake_thread)
```

5.2 2. Mutex

Q.15 : Utilité des mutex (avec schéma)

Un **mutex** (*MUTual EXclusion*) est un mécanisme de synchronisation qui garantit qu'un **seul thread** à la fois peut accéder à une **section critique** (ressource partagée).

Schéma du fonctionnement d'un mutex :

```
1 Thread A Mutex (libre) Thread B
2
3 mutex_lock() verrouill (owner=A)
4
5 Section Critique
6 accs la ressource mutex_lock()
7 partage (ex: var++) BLOQU
8 (ajout la file
9 d'attente du mutex)
10 mutex_unlock()
11 verrouill (owner=B)
12 RVEILL reprend
13 Section Critique
14 accs la ressource
15
16 mutex_unlock()
17 libre
```

Sans mutex : deux threads modifiant une même variable en parallèle produisent des **data races** (résultats incohérents). Par exemple, si deux threads font `compteur++` en même temps :

```
1 Thread A: lit compteur (=5) Thread B: lit compteur (=5)
2 Thread A: calcule 5+1=6 Thread B: calcule 5+1=6
3 Thread A: crit compteur=6 Thread B: crit compteur=6
4 Rsltat : compteur=6 au lieu de 7 !
```

Avec mutex : un seul thread accède à la variable à la fois → résultat correct.

Structure du mutex dans mthread (définie dans `include/mthread.h`) :

```
1 typedef struct mthread_mutex_s {
2     bool is_initialized = false; // mutex initialis ?
3     atomic_bool is_locked = false; // mutex verrouill ? (atomique)
4     struct mthread_thread_s *owner = nullptr; // thread propriétaire
5     atomic_flag thread_list_spinlock = ATOMIC_FLAG_INIT; // spinlock de protection
6     mthread_list_t thread_list; // file d'attente des threads bloques
7 } mthread_mutex_t;
```

5.3 3. Mise en place des mutex dans mthread

Q.16 : Fonction `mthread_mutex_init` — Localisation et implémentation

Fichier source : `mthread/src/mthread_mutex.cpp` (ligne 108)

Rôle : Initialise un mutex avant sa première utilisation.

Code source :

```
1 // src/mthread_mutex.cpp mthread_mutex_init (ligne 108)
2 int mthread_mutex_init(mthread_mutex_attr_t *attr, mthread_mutex_t *lock) {
3     if (lock == nullptr) {
4         return MTHREAD_MUTEX_ERROR_NULL;
5     }
6     if (attr != nullptr) {
7         not_implemented(); // les attributs ne sont pas encore supports
8     }
```



```

9  if (lock->is_initialized) {
10     return MTHREAD_MUTEX_ERROR_INIT; // dj initialis
11 }
12 { lock->is_initialized = true; }
13 return 0;
14 }

```

Détail de l'implémentation :

1. **Vérification du pointeur** : si `lock == nullptr` → retourne `MTHREAD_MUTEX_ERROR_NULL`.
2. **Attributs** : si des attributs sont fournis (`attr != nullptr`), la fonction appelle `not_implemented()` car les attributs ne sont pas encore supportés.
3. **Double initialisation** : si le mutex est déjà initialisé (`is_initialized == true`) → retourne `MTHREAD_MUTEX_ERROR_INIT` (protection contre la réinitialisation).
4. **Initialisation** : met simplement `is_initialized = true`. Les autres champs (`is_locked`, `owner`, `thread_list`) gardent leurs valeurs par défaut (grâce aux initialisateurs C++ dans la structure).

Note : L'initialisation est **minimaliste** car la structure `pthread_mutex_t` a des valeurs par défaut en C++ (`is_locked = false`, `owner = nullptr`, etc.). La fonction ne fait que marquer le mutex comme valide.

Q.17 : Fonction `pthread_mutex_lock` — Localisation et implémentation

Fichier source : `pthread/src/pthread_mutex.cpp` (ligne 7)

Rôle : Verrouille un mutex. Si le mutex est déjà pris, le thread appelant est **bloqué** jusqu'à ce que le mutex soit libéré.

Code source :

```

1  // src/pthread_mutex.cpp pthread_mutex_lock (ligne 7)
2  int pthread_mutex_lock(pthread_mutex_t *lock) {
3      bool unlocked = false;
4      if (lock == nullptr) {
5          return MTHREAD_MUTEX_ERROR_NULL;
6      } else {
7          // Auto-initialisation si ncessaire
8          if (!lock->is_initialized) {
9              int res = pthread_mutex_init(nullptr, lock);
10             if (res != 0) return MTHREAD_MUTEX_ERROR_INIT;
11         }
12
13         pthread_t *current_thread = pthread_self();
14
15         // Prendre le spinlock pour protger l'accs au mutex
16         pthread_spin_lock(&(lock->thread_list_spinlock));
17
18         if (!lock->is_locked) {
19             // CAS 1 : Mutex libre le prendre
20             lock->is_locked = true;
21             lock->owner = current_thread;
22             // Le spinlock sera libré implicitement (retour avant yield)
23             return 0;
24         } else {
25             // CAS 2 : Mutex dj pris se bloquer
26             pthread_list_item_t item;
27             item.thread = current_thread;
28             // Ajouter le thread la file d'attente du mutex
29             insert_tail_in_list(&item, &lock->thread_list);

```

```

30 // Se bloquer (libre le spinlock + yield)
31 pthread_block_self(lock);
32 }
33 }
34 return 0;
35 }

```

Détail de l'implémentation :

1. **Vérification** : pointeur null + auto-initialisation si nécessaire.
2. **Acquisition du spinlock** : `pthread_internal_spin_lock()` protège l'accès concurrent à la structure du mutex (busy-waiting atomique avec `atomic_flag_test_and_set`).
3. **Cas 1 — Mutex libre** (`!lock->is_locked`) :
 - Met `is_locked = true`
 - Enregistre le thread courant comme `owner`
 - Retourne immédiatement
4. **Cas 2 — Mutex pris** :
 - Crée un `pthread_list_item_t` contenant le thread courant
 - L'insère en `queue` de la `thread_list` du mutex (FIFO)
 - Appelle `pthread_block_self(lock)` (statut `pthread_blocked` + `yield`)
 - À son réveil, le thread reprend ici et retourne 0

Remarque : dans ce code, il y a un **bug subtil** : dans le cas *mutex libre*, le spinlock n'est pas libéré avant le `return 0`. Il devrait y avoir un `pthread_internal_spin_unlock(&(lock->thread_list_spinlock))` juste avant de retourner.

Q.18 : Fonction `pthread_mutex_unlock` — Localisation et implémentation

Fichier source : `pthread/src/pthread_mutex.cpp` (ligne 43)

Rôle : Déverrouille un mutex. S'il y a des threads en attente, réveille le premier de la file.

Code source :

```

1 // src/pthread_mutex.cpp pthread_mutex_unlock (ligne 43)
2 int pthread_mutex_unlock(pthread_mutex_t *lock) {
3     bool unlocked = false;
4     if (lock == nullptr) {
5         return PTHREAD_MUTEX_ERROR_NULL;
6     } else {
7         // Auto-initialisation si ncessaire
8         if (!lock->is_initialized) {
9             int res = pthread_mutex_init(nullptr, lock);
10            if (res != 0) return PTHREAD_MUTEX_ERROR_INIT;
11        }
12        // Vrifier que le mutex est bien verrouill
13        if (!atomic_load(&(lock->is_locked))) {
14            return PTHREAD_MUTEX_ERROR_NOT_LOCKED;
15        }
16        // Vrifier que c'est bien le proprietaire qui unlock
17        current_thread = pthread_self();
18        if (current_thread != lock->owner) {
19            return PTHREAD_MUTEX_ERROR_OWNER;
20        }
21
22        if (lock->thread_list.head != nullptr) {
23            // CAS 1 : Des threads attendent rveiller le premier
24            pthread_internal_spin_lock(&(lock->thread_list_spinlock));
25            pthread_list_item_t *item = remove_head_in_list(&(lock->thread_list));
26            pthread_internal_spin_unlock(&(lock->thread_list_spinlock));

```

```

27         // Transférer la propriété au thread réveillé
28         lock->owner = item->thread;
29         mthread_wake_thread(item->thread);
30     } else {
31         // CAS 2 : Personne n'attend libérer le mutex
32         atomic_store(&(lock->is_locked), unlocked); // is_locked = false
33     }
34 }
35 }
36 return 0;
37 }

```

Détail de l'implémentation :

1. Vérifications :

- Pointeur null → `MTHREAD_MUTEX_ERROR_NULL`
- Mutex non verrouillé → `MTHREAD_MUTEX_ERROR_NOT_LOCKED`
- Thread appelant $\neq$ propriétaire → `MTHREAD_MUTEX_ERROR_OWNER`

2. Cas 1 — Des threads en attente (`thread_list.head != nullptr`) :

- Prend le spinlock pour protéger la file
- Retire le **premier thread** (`remove_head_in_list` → FIFO)
- Libère le spinlock
- **Transfère la propriété** du mutex au thread réveillé (`lock->owner = item->thread`)
- **Réveille** le thread via `mthread_wake_thread()`
- Le mutex reste **verrouillé** : il change de propriétaire

3. Cas 2 — Personne n'attend :

- Met `is_locked = false` (via `atomic_store`)
- Le prochain thread pourra l'acquiescer directement

Schéma du transfert de propriété lors du unlock :

```

1 File d'attente du mutex : [Thread B] [Thread C] [Thread D]
2
3 Thread A appelle unlock() :
4   1. Retire Thread B de la file
5   2. owner = Thread B (transfert)
6   3. wake_thread(B) B reprend dans lock() et retourne 0
7
8 File d'attente restante : [Thread C] [Thread D]
9 Mutex toujours verrouillé, owner = Thread B

```

Q.19 : Fonction `mthread_mutex_destroy` — Localisation et implémentation

La fonction `mthread_mutex_destroy` n'existait pas dans le code fourni. Elle n'était ni déclarée dans `include/mthread.h`, ni implémentée dans `src/mthread_mutex.cpp`. Il a fallu la créer.

Fichier source : `mthread/src/mthread_mutex.cpp` (ajoutée en fin de fichier)

Déclaration : `mthread/include/mthread.h` (ajoutée après `mthread_mutex_init`)

Rôle : Détruire un mutex — opération inverse de `mthread_mutex_init()`. Elle remet la structure dans un état non initialisé, en libérant logiquement la ressource.

Code source :

```

1 // src/mthread_mutex.cpp mthread_mutex_destroy
2 int mthread_mutex_destroy(mthread_mutex_t *lock) {
3     if (lock == nullptr) {
4         return MTHREAD_MUTEX_ERROR_NULL;
5     }
6     if (!lock->is_initialized) {

```

```

7   return MTHREAD_MUTEX_ERROR_INIT;
8   }
9   if (atomic_load(&(lock->is_locked))) {
10      return MTHREAD_MUTEX_ERROR_ALREADY_LOCKED;
11   }
12   if (lock->thread_list.head != nullptr) {
13      return MTHREAD_MUTEX_ERROR_ALREADY_LOCKED;
14   }
15   lock->is_initialized = false;
16   lock->is_locked = false;
17   lock->owner = nullptr;
18   lock->thread_list.head = nullptr;
19   lock->thread_list.tail = nullptr;
20   return 0;
21 }

```

Détail de l'implémentation :

1. **Vérification du pointeur NULL** : si `lock == nullptr` → retourne `MTHREAD_MUTEX_ERROR_NULL`.
2. **Vérification de l'initialisation** : si le mutex n'a jamais été initialisé (`!is_initialized`) → retourne `MTHREAD_MUTEX_ERROR_INIT`.
3. **Vérification du verrouillage** : test atomique `atomic_load(&(lock->is_locked))`. Détruire un mutex verrouillé est un comportement indéfini en POSIX ; ici on retourne une erreur.
4. **Vérification de la file d'attente** : si `thread_list.head != nullptr`, des threads attendent → on refuse la destruction.
5. **Réinitialisation** : remise des champs à leurs valeurs par défaut (`is_initialized=false`, `owner=nullptr`, file vidée).

Comparaison avec `pthread_mutex_destroy` (POSIX) :

- POSIX : détruire un mutex verrouillé ou avec des threads en attente → **comportement indéfini**.
- Ici : implémentation **plus sûre** (codes d'erreur explicites).

Q.20 : Fonction `pthread_mutex_trylock` — Implémentation et explication

Fichier source : `pthread/src/pthread_mutex.cpp` (ligne 84)

Rôle : Tentative **non bloquante** de verrouillage d'un mutex. Si le mutex est libre, il est acquis. S'il est déjà pris, la fonction retourne **immédiatement** avec un code d'erreur, sans bloquer le thread appelant.

Code source :

```

1 // src/pthread_mutex.cpp pthread_mutex_trylock (ligne 84)
2 int pthread_mutex_trylock(pthread_mutex_t *lock) {
3     bool unlocked = false;
4     if (lock == nullptr) {
5         return MTHREAD_MUTEX_ERROR_NULL;
6     } else {
7         if (!lock->is_initialized) {
8             int res;
9             res = pthread_mutex_init(nullptr, lock);
10            if (res != 0) {
11                return MTHREAD_MUTEX_ERROR_INIT;
12            }
13        }
14        assert(lock != nullptr);
15        assert(lock->is_initialized == true);
16        if (atomic_compare_exchange_strong(&(lock->is_locked), &unlocked, true)) {

```

```

17     lock->owner = pthread_self();
18     pthread_log("MUTEX", "TryLock %p hold %p\n", lock->owner, lock);
19     return 0;
20 } else {
21     return MTHREAD_MUTEX_ERROR_ALREADY_LOCKED;
22 }
23 }
24 return 0;
25 }

```

Détail de l'implémentation :

1. **Vérifications standard** (comme `lock`) :
 - Pointeur null → `MTHREAD_MUTEX_ERROR_NULL`
 - Auto-initialisation si `!is_initialized`
2. **Tentative atomique avec `atomic_compare_exchange_strong`** :
 - On prépare `unlocked = false` (valeur attendue dans `is_locked`)
 - L'opération CAS fait atomiquement :
 - Compare `lock->is_locked` avec `unlocked (= false)`
 - Si `is_locked == false` : met `is_locked = true` → succès
 - Si `is_locked == true` : ne modifie rien → échec
3. **Cas succès** (CAS réussi) :
 - Enregistre le thread courant comme propriétaire : `lock->owner = pthread_self()`
 - Retourne 0
4. **Cas échec** (mutex déjà verrouillé) :
 - Retourne `MTHREAD_MUTEX_ERROR_ALREADY_LOCKED`
 - **Aucun blocage**, aucune insertion dans une file d'attente

Différence fondamentale avec `pthread_mutex_lock` :

Aspect	lock	trylock
Mutex libre	Acquis ✓	Acquis ✓
Mutex pris	Thread bloqué (yield)	Retourne immédiatement avec erreur
Utilise spinlock	Oui (<code>pthread_spin_lock</code>)	Non (CAS atomique)
Utilise file d'attente	Oui (<code>insert_tail_in_list</code>)	Non
Appelle <code>pthread_block_self</code>	Oui	Non

Pourquoi `trylock` n'utilise pas de spinlock ? Parce que `trylock` ne modifie jamais la `thread_list`. Le CAS atomique sur `is_locked` suffit pour garantir qu'un seul thread acquiert le mutex à la fois.

Cas d'usage typique :

```

1 // Polling : essayer d'acquies le mutex sans bloquer
2 if (pthread_mutex_trylock(&m) == 0) {
3     // Section critique le mutex est acquis
4     shared_resource++;
5     pthread_mutex_unlock(&m);
6 } else {
7     // Le mutex est pris faire autre chose en attendant
8     do_other_work();
9 }

```

Q.21 : Macro `MTHREAD_MUTEX_INITIALIZER` — Implémentation et explication

Fichier : `pthread/include/pthread.h` (ajoutée après la définition de `pthread_mutex_t`)

Rôle : Permettre l'initialisation **statique** d'un mutex, sans appeler `pthread_mutex_init()`.

Implémentation :

```
1 // include/pthread.h après la définition de pthread_mutex_t
2 #define PTHREAD_MUTEX_INITIALIZER { \
3     .is_initialized = true, \
4     .is_locked = false, \
5     .owner = nullptr, \
6     .thread_list_spinlock = ATOMIC_FLAG_INIT, \
7     .thread_list = {nullptr, nullptr} \
8 }
```

Explication détaillée : c'est un **designated initializer** C/C++ qui initialise chaque champ de `pthread_mutex_t` :

Champ	Valeur	Signification
<code>is_initialized</code>	<code>true</code>	Mutex prêt à l'emploi
<code>is_locked</code>	<code>false</code>	Mutex libre
<code>owner</code>	<code>nullptr</code>	Aucun propriétaire
<code>thread_list_spinlock</code>	<code>ATOMIC_FLAG_INIT</code>	Spinlock initialisé (déverrouillé)
<code>thread_list</code>	<code>{nullptr, nullptr}</code>	File d'attente vide

Utilisation :

```
1 // AVANT (initialisation dynamique) :
2 pthread_mutex_t m;
3 pthread_mutex_init(nullptr, &m);
4
5 // APRÈS (initialisation statique) :
6 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
```

Avantages de la macro :

1. **Global/statiques :** permet l'init au niveau fichier (avant `main`) :

```
1 // Au niveau fichier (global)
2 static pthread_mutex_t global_mutex = PTHREAD_MUTEX_INITIALIZER;
```

2. **Pas de risque d'oubli :** mutex prêt dès la déclaration.

3. **Compatibilité POSIX :** équivalent de `PTHREAD_MUTEX_INITIALIZER`.

Note : Avec cette macro, la branche d'auto-initialisation dans `lock/unlock/trylock` n'est normalement jamais prise car `is_initialized` vaut déjà `true`.

Q.22 : Programmes de test pour `mutex_destroy`, `trylock` et `PTHREAD_MUTEX_INITIALIZER`

Fichier source : `pthread/tests/test_scheduler_unit_mutex_destroy.cpp`

Programme de test complet :

```
1 #include <pthread.h>
2 #include <cassert>
3 #include <cstdio>
4
5 static int shared_counter = 0;
6 static pthread_mutex_t mutex_lock;
7
8 // Mutex initialisé statiquement (Q.21)
```

```

9  static pthread_mutex_t static_mutex = PTHREAD_MUTEX_INITIALIZER;
10
11  // TEST Q.19 : pthread_mutex_destroy
12  void test_mutex_destroy() {
13      printf("=== Test pthread_mutex_destroy ===\n");
14
15      pthread_mutex_t m;
16      int res;
17
18      // Test 1 : Initialiser puis détruire un mutex libre
19      res = pthread_mutex_init(&m, NULL);
20      assert(res == 0);
21      res = pthread_mutex_destroy(&m);
22      assert(res == 0);
23      printf(" [OK] destroy sur mutex non verrouillé\n");
24
25      // Test 2 : destroy(NULL) erreur
26      res = pthread_mutex_destroy(NULL);
27      assert(res == PTHREAD_MUTEX_ERROR_NULL);
28      printf(" [OK] destroy(NULL) PTHREAD_MUTEX_ERROR_NULL\n");
29
30      // Test 3 : destroy sur mutex non initialisé erreur
31      pthread_mutex_t m2;
32      m2.is_initialized = false;
33      res = pthread_mutex_destroy(&m2);
34      assert(res == PTHREAD_MUTEX_ERROR_INIT);
35      printf(" [OK] destroy sur mutex non initialisé PTHREAD_MUTEX_ERROR_INIT\n");
36
37      printf("=== Test pthread_mutex_destroy : PASS ===\n\n");
38  }
39
40  // TEST Q.20 : pthread_mutex_trylock
41  static void *trylock_thread_func(void *arg) {
42      // Le mutex est déjà verrouillé par le thread principal
43      int res = pthread_mutex_trylock(&mutex_lock);
44      if (res == 0) {
45          shared_counter++;
46          printf(" Thread secondaire : trylock réussi, counter = %d\n", shared_counter);
47          pthread_mutex_unlock(&mutex_lock);
48      } else {
49          printf(" Thread secondaire : trylock échoué %d (ALREADY_LOCKED)\n", res);
50          assert(res == PTHREAD_MUTEX_ERROR_ALREADY_LOCKED);
51      }
52      return NULL;
53  }
54
55  void test_mutex_trylock() {
56      printf("=== Test pthread_mutex_trylock ===\n");
57
58      int res;
59      res = pthread_mutex_init(&mutex_lock, NULL);
60      assert(res == 0);
61
62      // Test 1 : trylock sur mutex libre succès
63      res = pthread_mutex_trylock(&mutex_lock);
64      assert(res == 0);
65      printf(" [OK] trylock sur mutex libre succès\n");
66
67      // Test 2 : un autre thread tente trylock sur mutex verrouillé
68      pthread_t *t1;

```

```

69  pthread_create_thread(&t1, trylock_thread_func, nullptr);
70  pthread_mutex_unlock(&mutex_lock);
71  pthread_join(t1, nullptr);
72  printf(" [OK] trylock depuis un autre thread\n");
73
74  // Test 3 : trylock(NULL) erreur
75  res = pthread_mutex_trylock(nullptr);
76  assert(res == MTHREAD_MUTEX_ERROR_NULL);
77  printf(" [OK] trylock(NULL) MTHREAD_MUTEX_ERROR_NULL\n");
78
79  pthread_mutex_destroy(&mutex_lock);
80  printf("=== Test pthread_mutex_trylock : PASS ===\n\n");
81 }
82
83 // TEST Q.21 : MTHREAD_MUTEX_INITIALIZER
84 static void *static_mutex_thread_func(void *arg) {
85     pthread_mutex_lock(&static_mutex);
86     shared_counter++;
87     printf(" Thread : counter = %d (via MTHREAD_MUTEX_INITIALIZER)\n",
88           shared_counter);
89     pthread_mutex_unlock(&static_mutex);
90     return nullptr;
91 }
92
93 void test_mutex_initializer() {
94     printf("=== Test MTHREAD_MUTEX_INITIALIZER ===\n");
95     shared_counter = 0;
96
97     // Test 1 : lock/unlock direct sans appeler init
98     pthread_mutex_lock(&static_mutex);
99     shared_counter = 42;
100    printf(" [OK] lock/unlock sans init, counter = %d\n", shared_counter);
101    pthread_mutex_unlock(&static_mutex);
102
103    // Test 2 : 5 threads concurrents sur le mutex statique
104    shared_counter = 0;
105    const int NB = 5;
106    pthread_thread_t *threads[NB];
107    for (int i = 0; i < NB; i++)
108        pthread_create_thread(&threads[i], static_mutex_thread_func, nullptr);
109    for (int i = 0; i < NB; i++)
110        pthread_join(threads[i], nullptr);
111
112    assert(shared_counter == NB);
113    printf(" [OK] %d threads counter = %d (exclusion mutuelle OK)\n",
114          NB, shared_counter);
115
116    pthread_mutex_destroy(&static_mutex);
117    printf("=== Test MTHREAD_MUTEX_INITIALIZER : PASS ===\n\n");
118 }
119
120 // MAIN
121 int main() {
122     pthread_init_scheduler_fifo(1);
123
124     test_mutex_destroy();
125     test_mutex_trylock();
126     test_mutex_initializer();
127
128     printf("=====\n");

```



```

129     printf(" TOUS LES TESTS SONT PASSES !\n");
130     printf("=====\n");
131     return 0;
132 }

```

Compilation et exécution :

```

1 cd mthread/build
2 cmake ..
3 make test_scheduler_unit_mutex_destroy
4 ./tests/test_scheduler_unit_mutex_destroy

```

Sortie attendue :

```

1 === Test mthread_mutex_destroy ===
2 [OK] destroy sur mutex non verrouill
3 [OK] destroy(NULL) MTHREAD_MUTEX_ERROR_NULL
4 [OK] destroy sur mutex non initialis MTHREAD_MUTEX_ERROR_INIT
5 === Test mthread_mutex_destroy : PASS ===
6
7 === Test mthread_mutex_trylock ===
8 [OK] trylock sur mutex libre succs
9 Thread secondaire : trylock chou 5 (ALREADY_LOCKED)
10 [OK] trylock depuis un autre thread
11 [OK] trylock(NULL) MTHREAD_MUTEX_ERROR_NULL
12 === Test mthread_mutex_trylock : PASS ===
13
14 === Test MTHREAD_MUTEX_INITIALIZER ===
15 [OK] lock/unlock sans init, counter = 42
16 Thread : counter = 1 (via MTHREAD_MUTEX_INITIALIZER)
17 Thread : counter = 2 (via MTHREAD_MUTEX_INITIALIZER)
18 Thread : counter = 3 (via MTHREAD_MUTEX_INITIALIZER)
19 Thread : counter = 4 (via MTHREAD_MUTEX_INITIALIZER)
20 Thread : counter = 5 (via MTHREAD_MUTEX_INITIALIZER)
21 [OK] 5 threads counter = 5 (exclusion mutuelle OK)
22 === Test MTHREAD_MUTEX_INITIALIZER : PASS ===
23
24 =====
25 TOUS LES TESTS SONT PASSES !
26 =====

```

Résumé des tests :

Question	Fonction/Macro testée	Tests effectués	Résultat
Q.19	<code>mthread_mutex_destroy</code>	destroy libre ✓, NULL ✓, non-init ✓	PASS
Q.20	<code>mthread_mutex_trylock</code>	mutex libre ✓, mutex pris ✓, NULL ✓	PASS
Q.21	<code>MTHREAD_MUTEX_INITIALIZER</code>	lock/unlock sans init ✓, 5 threads concurrents ✓	PASS